



Programación concurrente

Fundamentos de Lenguajes de Programación – IIC2560

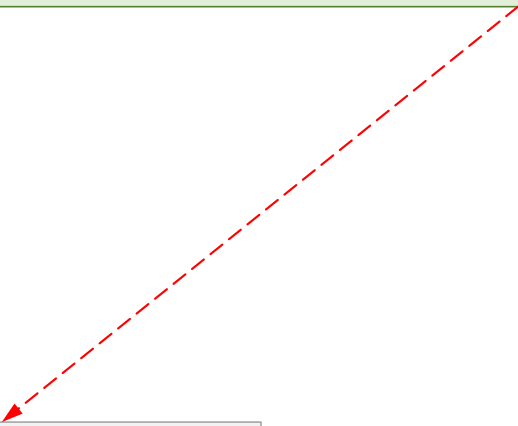
Yadran Eterovic S. (yadran@uc.cl)

En la ejecución de un programa visualizamos el flujo de control imaginando una hebra (*thread*) que va uniendo todos los comandos ejecutados en el orden en que son ejecutados:

- una hebra (de control) identifica la secuencia de comandos (de un programa) ejecutada en una computación particular

En la ejecución de un programa visualizamos el flujo de control imaginando una hebra (*thread*) que va uniendo todos los comandos ejecutados en el orden en que son ejecutados:

- una hebra (de control) identifica la secuencia de comandos (de un programa) ejecutada en una computación particular



Programa *secuencial*: Sólo una hebra está activa en cualquier momento de la ejecución

En la ejecución de un programa visualizamos el flujo de control imaginando una hebra (*thread*) que va uniendo todos los comandos ejecutados en el orden en que son ejecutados:

- una hebra (de control) identifica la secuencia de comandos (de un programa) ejecutada en una computación particular

Programa *concurrente*: Varias hebras —procesos, tareas (*tasks*) o programas secuenciales— activas al mismo tiempo

Programa *secuencial*: Sólo una hebra está activa en cualquier momento de la ejecución

Distinguimos *concurrency física* de *concurrency lógica*

Distinguimos *concurrency física* de *concurrency lógica*

Ejecución simultánea, o en paralelo, de varios threads:

- p.ej., en computadores con varios procesadores

Distinguimos *concurrency física* de *concurrency lógica*

Ejecución simultánea, o en paralelo, de varios threads:

- p.ej., en computadores con varios procesadores

El programador y la aplicación perciben la ejecución como concurrente
... pero, físicamente, las actividades concurrentes son mapeadas a ejecuciones secuenciales

Distinguimos *concurrency física* de *concurrency lógica*

Ejecución simultánea, o en paralelo, de varios threads:

- p.ej., en computadores con varios procesadores

El programador y la aplicación perciben la ejecución como concurrente

... pero, físicamente, las actividades concurrentes son mapeadas a ejecuciones secuenciales

Conceptualmente y desde el punto de vista de los lenguajes de programación, esta distinción no es muy relevante ... excepto si consideramos el desempeño del sistema y el tiempo de ejecución

Concurrencia física

Ejecución simultánea, o en paralelo, de varios threads:

- p.ej., en computadores con varios procesadores



Programación paralela

Métodos y algoritmos dedicados para definir soluciones paralelas eficientes principalmente a problemas de computación científica:

- aplicación de la misma operación a datos diferentes
- aplicación de operaciones diferentes en paralelo a los mismos datos
- e.g., solución de ecuaciones diferenciales

Concurrencia física

Ejecución simultánea, o en paralelo, de varios threads:

- p.ej., en computadores con varios procesadores



Programación paralela

Métodos y algoritmos dedicados para definir soluciones paralelas eficientes principalmente a problemas de computación científica:

- aplicación de la misma operación a datos diferentes
- aplicación de operaciones diferentes en paralelo a los mismos datos
- e.g., solución de ecuaciones diferenciales



Desde el punto de vista lingüístico no presenta problemas diferentes de aquellos de la concurrencia lógica

Concurrencia lógica
... o simplemente *concurrencia*

Programación *multihebras*

Presencia de más de una hebra (o proceso) —más de un contexto de ejecución activo al mismo tiempo

La aplicación corre en un computador individual

... ya sea, un único procesador

... o un multiprocesador con memoria compartida

Concurrencia lógica
... o simplemente *concurrencia*

Programación *multihebras*

Presencia de más de una hebra (o proceso) —más de un contexto de ejecución activo al mismo tiempo

La aplicación corre en un computador individual
... ya sea, un único procesador
... o un multiprocesador con memoria compartida

Programación *distribuida*

Programas concurrentes diseñados para correr en máquinas hechas de estructuras físicas distribuidas:

- multicomputadores con memoria distribuida
- redes de computadores con arquitecturas y topologías diversas

Los nodos que efectúan la computación están físicamente distribuidos:

- no pueden compartir su memoria
- requieren primitivas de comunicación y mecanismos apropiados basados en intercambio de mensajes

Curso IIC2523

Concurrencia lógica

... o simplemente *concurrency*Programación *multihebras*

Presencia de más de una hebra (o proceso) —más de un contexto de ejecución activo al mismo tiempo

La aplicación corre en un computador individual

... ya sea, un único procesador

... o un multiprocesador con memoria compartida

Dos threads, *T1* y *T2*, ambos quieren incrementar el valor de la variable compartida *x*

<i>T1</i>		read <i>x</i> in reg1
<i>T2</i>		read <i>x</i> in reg2
<i>T2</i>		increment reg2
<i>T2</i>		write reg2 in <i>x</i>
<i>T1</i>		increment reg1
<i>T1</i>		write reg1 in <i>x</i>

Programación *distribuida*

Programas concurrentes diseñados para correr en máquinas hechas de estructuras físicas distribuidas:

- multicomputadores con memoria distribuida
- redes de computadores con arquitecturas y topologías diversas

Los nodos que efectúan la computación están físicamente distribuidos:

- no pueden compartir su memoria
- requieren primitivas de comunicación y mecanismos apropiados basados en intercambio de mensajes

Curso IIC2523

Programación concurrente
(multihebras o *otherwise*)

Dos temas principales, independientes del tipo de concurrencia y comunes a lenguajes y modelos

Programación concurrente
(multihebras o *otherwise*)

Dos temas principales, independientes del tipo de concurrencia y comunes a lenguajes y modelos

Comunicación

Se necesita mecanismos que permitan el intercambio de información entre las diversas actividades que contribuyen a la realización de alguna función:

- e.g., resultados parciales de una computación, información de control

Si la comunicación es entre dos procesos, distinguimos *emisor* y *receptor*

Conceptualmente, hay dos mecanismos principales:

- *memoria compartida*
- *envío y recepción* (i.e., intercambio) *de mensajes*

Programación concurrente
(multihebras o *otherwise*)

Dos temas principales, independientes del tipo de concurrencia y comunes a lenguajes y modelos

Comunicación

Se necesita mecanismos que permitan el intercambio de información entre las diversas actividades que contribuyen a la realización de alguna función:

- e.g., resultados parciales de una computación, información de control

Si la comunicación es entre dos procesos, distinguimos *emisor* y *receptor*

Conceptualmente, hay dos mecanismos principales:

- *memoria compartida*
- *envío y recepción* (i.e., intercambio) *de mensajes*

Programación multihebras

Programación distribuida

Programación concurrente
(multihebras o *otherwise*)

Dos temas principales, independientes del tipo de concurrencia y comunes a lenguajes y modelos

Comunicación

Se necesita mecanismos que permitan el intercambio de información entre las diversas actividades que contribuyen a la realización de alguna función:

- e.g., resultados parciales de una computación, información de control

Si la comunicación es entre dos procesos, distinguimos *emisor* y *receptor*

Conceptualmente, hay dos mecanismos principales:

- *memoria compartida*
- *envío y recepción* (i.e., intercambio) *de mensajes*

Programación multihebras

Programación distribuida

Sincronización

Mecanismo que controla el orden en que se ejecutan los procesos

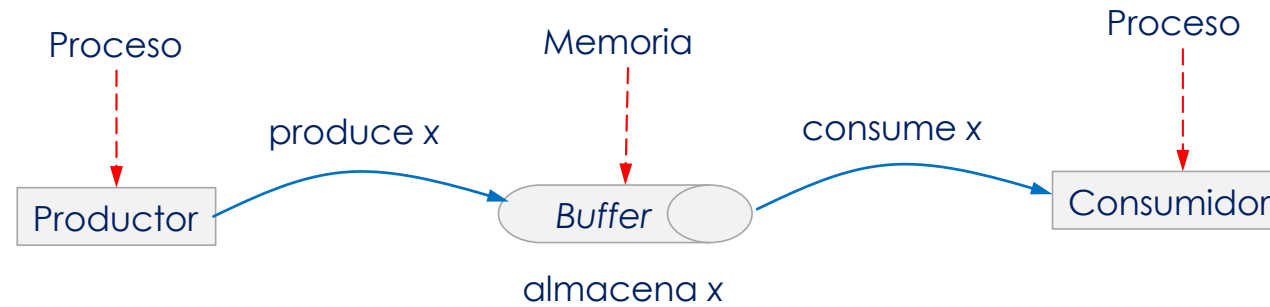
Lo que hemos visto
en el curso hasta aquí

Memoria compartida : Dos partes de un mismo programa tienen acceso a la misma variable compartida

→ se comunican simplemente escribiendo un valor en la variable y leyendo ese valor

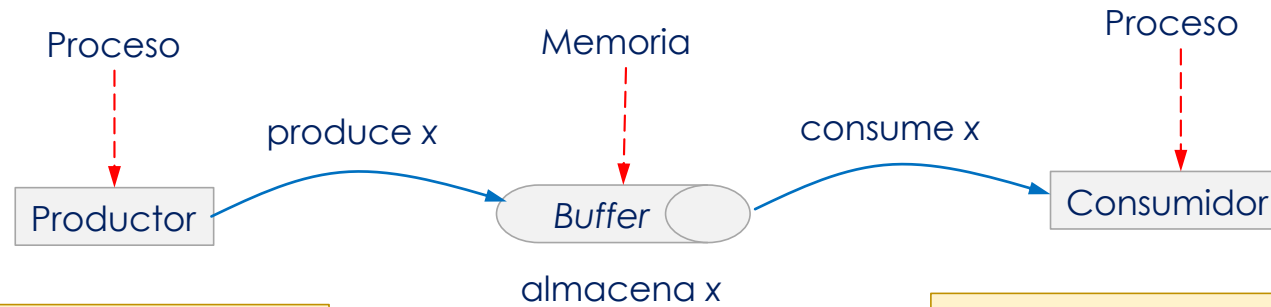
Lo que hemos visto
en el curso hasta aquí

Memoria compartida : Dos partes de un mismo programa tienen acceso a la misma variable compartida
→ se comunican simplemente escribiendo un valor en la variable y leyendo ese valor



Lo que hemos visto
en el curso hasta aquí

Memoria compartida : Dos partes de un mismo programa tienen acceso a la misma variable compartida
→ se comunican simplemente escribiendo un valor en la variable y leyendo ese valor



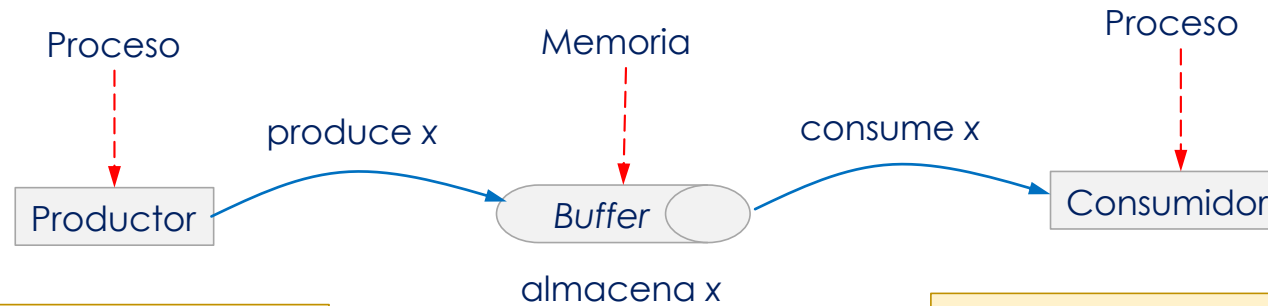
Debe evitar tener acceso al *buffer* al mismo tiempo que el *Consumidor*

Un tipo de sincronización

Debe evitar tener acceso al *buffer* al mismo tiempo que el *Productor*

Lo que hemos visto
en el curso hasta aquí

Memoria compartida : Dos partes de un mismo programa tienen acceso a la misma variable compartida
→ se comunican simplemente escribiendo un valor en la variable y leyendo ese valor



Debe evitar tener acceso al *buffer* al mismo tiempo que el *Consumidor*

Debe esperar hasta que el *buffer* esté vacío para producir (escribir) un nuevo (valor en) *x*
En cuanto lo produce, el *buffer* queda lleno

Un tipo de sincronización

Otro tipo de sincronización

Debe evitar tener acceso al *buffer* al mismo tiempo que el *Productor*

Debe esperar hasta que el *buffer* esté lleno para consumir (leer) un nuevo (valor de) *x*
En cuanto lo consume, el *buffer* queda vacío

Sincronización

Mecanismo que controla el orden en que se ejecutan los threads
... esencial para asegurar la corrección de la computación

- E.g., si dos procesos que se comunican usando la variable compartida x , entonces el proceso que recibe debe leer el valor de x sólo después de que el proceso que envía ha escrito el valor correcto en x

Sincronización

Mecanismo que controla el orden en que se ejecutan los threads
... esencial para asegurar la corrección de la computación

- E.g., si dos procesos que se comunican usando la variable compartida x , entonces el proceso que recibe debe leer el valor de x sólo después de que el proceso que envía ha escrito el valor correcto en x

Sincronización de condición (o por cooperación)

Permite la suspensión de la ejecución de un proceso hasta que ocurra una condición apropiada

- E.g., el Productor debe esperar hasta que el *buffer* esté vacío

Sincronización

Mecanismo que controla el orden en que se ejecutan los threads
... esencial para asegurar la corrección de la computación

- E.g., si dos procesos que se comunican usando la variable compartida x , entonces el proceso que recibe debe leer el valor de x sólo después de que el proceso que envía ha escrito el valor correcto en x

Sincronización de condición (o por cooperación)

Permite la suspensión de la ejecución de un proceso hasta que ocurra una condición apropiada

- E.g., el Productor debe esperar hasta que el *buffer* esté vacío

Exclusión mutua (o sincronización por competición)

Garantiza que ciertas secciones críticas del código no son accesibles a varios procesos al mismo tiempo

- E.g., Productor y Consumidor deben evitar tener acceso al *buffer* al mismo tiempo

Sincronización

Mecanismo que controla el orden en que se ejecutan los threads
... esencial para asegurar la corrección de la computación

- E.g., si dos procesos que se comunican usando la variable compartida *x*, entonces el proceso que recibe debe leer el valor de *x* sólo después de que el proceso que envía ha escrito el valor correcto en *x*

Sincronización de condición (o por cooperación)

Permite la suspensión de la ejecución de un proceso hasta que ocurra una condición apropiada

- E.g., el Productor debe esperar hasta que el *buffer* esté vacío

Exclusión mutua (o sincronización por competición)

Garantiza que ciertas secciones críticas del código no son accesibles a varios procesos al mismo tiempo

- E.g., Productor y Consumidor deben evitar tener acceso al *buffer* al mismo tiempo

- *Busy waiting*
- Bloqueo

Busy waiting (o spinning)

“Espera ocupada”

... para tener acceso a una sección crítica o hasta que se cumpla una condición dada

El proceso que tiene que esperar ejecuta un ciclo en el cual evalúa continuamente una condición hasta que ésta se vuelva verdadera:

- durante la espera, la CPU está empleada activamente
- eficiente en computadores multiprocesadores
- e.g., *locks* y barreras

Busy waiting (o spinning)
“Espera ocupada”

... para tener acceso a una sección crítica o hasta que se cumpla una condición dada

El proceso que tiene que esperar ejecuta un ciclo en el cual evalúa continuamente una condición hasta que ésta se vuelva verdadera:

- durante la espera, la CPU está empleada activamente
- eficiente en computadores multiprocesadores
- e.g., *locks* y barreras

Locks (u “obtener el permiso exclusivo”)

```
process Pi {  
  non-critical section;  
  acquire_lock( B );  
  critical section;  
  release_lock( B );  
  non-critical section;  
}
```

Busy waiting (o spinning)
“Espera ocupada”

... para tener acceso a una sección crítica o hasta que se cumpla una condición dada

El proceso que tiene que esperar ejecuta un ciclo en el cual evalúa continuamente una condición hasta que ésta se vuelva verdadera:

- durante la espera, la CPU está empleada activamente
- eficiente en computadores multiprocesadores
- e.g., *locks* y *barreras*

Locks (u “obtener el permiso exclusivo”)

```
process Pi {  
  non-critical section;  
  acquire_lock( B );  
  critical section;  
  release_lock( B );  
  non-critical section;  
}
```

Barreras (o “esperar hasta que todos lleguen”)

```
process Pi {  
  while B true do {  
    code of task i;  
    wait for all other Pj to terminate;  
    // barrier  
  }  
}
```

Busy waiting (o spinning)
“Espera ocupada”

... para tener acceso a una sección crítica o hasta que se cumpla una condición dada

El proceso que tiene que esperar ejecuta un ciclo en el cual evalúa continuamente una condición hasta que ésta se vuelva verdadera:

- durante la espera, la CPU está empleada activamente
- eficiente en computadores multiprocesadores
- e.g., *locks* y barreras

Locks (u “obtener el permiso exclusivo”)

```
process Pi {  
  non-critical section;  
  acquire_lock( B );  
  critical section;  
  release_lock( B );  
  non-critical section;  
}
```

Barreras (o “esperar hasta que todos lleguen”)

```
process Pi {  
  while B true do {  
    code of task i;  
    wait for all other Pj to terminate;  
    // barrier  
  }  
}
```

Programación paralela

Locks (u “obtener el permiso exclusivo”)

```
process Pi {  
  non-critical section;  
  acquire_lock( B );  
  critical section;  
  release_lock( B );  
  non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **Pi** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Locks (u “obtener el permiso exclusivo”)

El proceso **P_i** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **P_i** libera el permiso

```
process Pi {  
    non-critical section;  
    → acquire_lock( B );  
    critical section;  
    → release_lock( B );  
    non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **P_i** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Locks (u “obtener el permiso exclusivo”)

Cada proceso tiene acceso sólo a sus variables locales

El proceso P_i tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

El proceso P_i pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso P_i libera el permiso

```
process Pi {
  non-critical section;
  → acquire_lock( B );
  critical section;
  → release_lock( B );
  non-critical section;
}
```

Dos threads, T_1 y T_2 , ambos quieren incrementar el valor de la variable compartida x

T_1		read x in reg1
T_2		read x in reg2
T_2		increment reg2
T_2		write reg2 in x
T_1		increment reg1
T_1		write reg1 in x

sección crítica

```
{
  read x in reg1;
  increment reg1;
  write reg1 in x;
}
```


El proceso **Pi** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **Pi** libera el permiso

```
process Pi {  
  non-critical section;  
  → acquire_lock( B );  
  critical section;  
  → release_lock( B );  
  non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **Pi** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

El proceso **P_i** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **P_i** libera el permiso

```
process Pi {  
    non-critical section;  
    → acquire_lock( B );  
    critical section;  
    → release_lock( B );  
    non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **P_i** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Una forma de asegurarse que el permiso otorgado sea exclusivo es que esté asociado a una variable boolean tal que:

- si es verdadera → el permiso lo tiene algún proceso (y no se puede otorgar)
- si es falsa → nadie tiene el permiso
... la variable se pone en verdadera
... y se le otorga el permiso al proceso que lo está solicitando

El proceso **P_i** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **P_i** libera el permiso

```
process Pi {  
  non-critical section;  
  → acquire_lock( B );  
  critical section;  
  → release_lock( B );  
  non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **P_i** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Una forma de asegurarse que el permiso otorgado sea exclusivo es que esté asociado a una variable boolean tal que:

- si es verdadera → el permiso lo tiene algún proceso (y no se puede otorgar)
 - si es falsa → nadie tiene el permiso
- ... la variable se pone en verdadera

... y se le otorga el permiso al proceso que lo está solicitando

Estas dos acciones deben ejecutarse **indivisiblemente**, como en la instrucción de máquina **test_and_set**

El proceso **Pi** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **Pi** libera el permiso

```
process Pi {  
    non-critical section;  
    → acquire_lock( B );  
    critical section;  
    → release_lock( B );  
    non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **Pi** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Una forma de asegurarse que el permiso otorgado sea exclusivo es que esté asociado a una variable boolean tal que:

- si es verdadera → el permiso lo tiene algún proceso (y no se puede otorgar)
 - si es falsa → nadie tiene el permiso
- ... la variable se pone en verdadera
... y se le otorga el permiso al proceso que lo está solicitando

```
bool test_and_set(bool lock){  
    < bool init = lock;  
    lock = true;  
    return init; >  
}
```

La operación **test_and_set** es atómica: < y >
La atomicidad está garantizada por el hardware

Estas dos acciones deben ejecutarse **indivisiblemente**, como en la instrucción de máquina **test_and_set**

El proceso **Pi** pide permiso para tener acceso exclusivo a la sección crítica ... y espera hasta que se lo conceden

El proceso **Pi** libera el permiso

```
process Pi {  
  non-critical section;  
  → acquire_lock( B );  
  critical section;  
  → release_lock( B );  
  non-critical section;  
}
```

Cada proceso tiene acceso sólo a sus variables locales

El proceso **Pi** tiene acceso a variables compartidas por todos los procesos

Cada proceso tiene acceso sólo a sus variables locales

Una forma de asegurarse que el permiso otorgado sea exclusivo es que esté asociado a una variable boolean tal que:

- si es verdadera → el permiso lo tiene algún proceso (y no se puede otorgar)
 - si es falsa → nadie tiene el permiso
- ... la variable se pone en verdadera
... y se le otorga el permiso al proceso que lo está solicitando

```
bool test_and_set(bool lock){  
  < bool init = lock;  
  lock = true;  
  return init;  
}
```

La operación **test_and_set** es atómica: < y >
La atomicidad está garantizada por el hardware

El proceso espera (*spinning*) hasta que le otorgan el permiso

Estas dos acciones deben ejecutarse **indivisiblemente**, como en la instrucción de máquina **test_and_set**

```
bool B = false,  
...  
void acquire_lock(bool B){  
  → while test_and_set(B)  
    do skip;  
}  
  
void release_lock(bool B){  
  B = false;  
}
```

```
bool B = false,  
...  
void acquire_lock(bool B){  
    while test_and_set(B)   
        do skip;  
}  
  
void release_lock(bool B){  
    B = false;  
}
```

```
bool test_and_set(bool lock){  
    < bool init = lock;  
    lock = true;  
    return init; >  
}
```

Propiedades

- A lo más uno de los procesos que estén solicitando acceso a la sección crítica lo conseguirá —*propiedad de seguridad*
- Si hay uno o más procesos solicitando acceso a la sección crítica, garantizadamente uno lo conseguirá —*propiedad de progreso*
- Los procesos tratando de tener acceso a la sección crítica realizan innumerables operaciones **write** a la variable compartida **lock**
→ competencia por memoria y uso del bus

```

bool B = false,
...
void acquire_lock(bool B){
    while test_and_set(B)
        do skip;
}

void release_lock(bool B){
    B = false;
}

```

```

bool test_and_set(bool lock){
    < bool init = lock;
    lock = true;
    return init; >
}

```

Propiedades

- A lo más uno de los procesos que estén solicitando acceso a la sección crítica lo conseguirá —*propiedad de seguridad*
- Si hay uno o más procesos solicitando acceso a la sección crítica, garantizadamente uno lo conseguirá —*propiedad de progreso*
- Los procesos tratando de tener acceso a la sección crítica realizan innumerables operaciones **write** a la variable compartida **lock**
 → competencia por memoria y uso del bus

```

void acquire_lock(bool B){
    while B do skip;
    while test_and_set(B) do {
        while B do skip;
    }
}

```

```

bool B = false,
...
void acquire_lock(bool B){
    while test_and_set(B)
        do skip;
}

void release_lock(bool B){
    B = false;
}

```

```

bool test_and_set(bool lock){
    < bool init = lock;
    lock = true;
    return init; >
}

```

Propiedades

- A lo más uno de los procesos que estén solicitando acceso a la sección crítica lo conseguirá —*propiedad de seguridad*
- Si hay uno o más procesos solicitando acceso a la sección crítica, garantizadamente uno lo conseguirá —*propiedad de progreso*
- Los procesos tratando de tener acceso a la sección crítica realizan innumerables operaciones **write** a la variable compartida **lock**
→ competencia por memoria y uso del bus

Propiedades +

- **No** se garantiza la *propiedad de justicia*
→ un proceso que quiera tener acceso a la sección crítica finalmente lo conseguirá

```

void acquire_lock(bool B){
    while B do skip;
    while test_and_set(B) do {
        while B do skip;
    }
}

```


¿Por qué programación concurrente?

Velocidad de ejecución, aprovechando el hardware paralelo: Múltiples procesadores o múltiples *cores* en un procesador

Captura la estructura lógica del problema: Servidores, aplicaciones gráficas

Distribución física: E.g., aplicaciones que corren a través de la Internet; aplicaciones embebidas, como los sistemas de control de un auto

Programas cuya velocidad de ejecución está limitada por operaciones de I/O o el ancho de banda de la memoria